
Stochastic Algorithms for PCA

Nitish Joshi

Department of Computer Science
Indian Institute of Technology, Bombay
joshinh@gmail.com

Abstract

PCA is an omnipresent tool used in areas such as machine learning and data analysis especially for dimensionality reduction. In this paper, we study and review various stochastic algorithms for PCA. We also present extensive comparison between these algorithms both theoretically as well as empirically.

1 Introduction

Principal Component Analysis (PCA) is one of the most common tools used in machine learning and statistics for unsupervised data analysis and preprocessing. It has tremendous applications and is useful in a broad range of areas from quantitative finance to neuroscience. The problem can be stated as follows: Given some random vector $x \in \mathbb{R}^d$ from an unknown distribution D , what is the k -dimensional subspace on which the projection has the largest possible variance. Formally we can write PCA as the following optimization problem:

$$\begin{aligned} \min_{U \in \mathbb{R}^{d \times k}} & -\mathbb{E}[\|U^T \mathbf{x}\|^2] \\ \text{s.t. } & U^T U = I_k \end{aligned} \tag{1}$$

where $U \in \mathbb{R}^{d \times k}$ is the orthogonal basis matrix that spans the subspace and I_k is the k -dimensional identity matrix. It can be easily shown that the maximal variance subspace is given by the span of the eigenvectors corresponding to the top- k eigenvalues of the co-variance matrix $C = \mathbb{E}[\mathbf{x}\mathbf{x}^T]$. Also, given n i.i.d samples from the distribution D , the empirical risk minimization problem reduces to finding the top- k eigenvectors of the empirical co-variance matrix, $\hat{C} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^T$.

Another approach for solving PCA is to directly optimize the objective function in (1) using stochastic approximation algorithms. Oja [1982] directly use stochastic gradient descent (SGD) on the objective function and the algorithm is commonly known as Oja's Algorithm. Since the PCA objective is non-convex, Arora et al. [2013] use convex relaxation of the problem followed by SGD, also referred to as the matrix stochastic gradient (MSG):

$$M_{t+1} = \mathcal{P}(M_t + \eta_t \mathbf{x}_t \mathbf{x}_t^T) \tag{2}$$

where $\mathcal{P}$ is projection onto a feasible set and M_t are the intermediate iterate for $M = UU^T$ in the objective function. This re-parameterization from U to M helps convert the objective to a convex function (linear in fact). Mianjy and Arora [2018] study various modifications of the MSG problem by adding l_1 , l_2 and an elastic net of both the regularizations, and analyze its advantages in terms of convergence and overall run-time.

2 Methods

In this section, we describe in detail the Oja's algorithm, incremental SVD and MSG and analyze their theoretical guarantees and computational trade-offs.

2.1 Oja's Algorithm

Although the PCA optimization problem is non-convex, gradient descent is a viable algorithm and the Oja's algorithm has been backed by strong theoretical guarantees Allen-Zhu and Li [2017]. Oja's algorithm directly minimizes the PCA objective function using the following stochastic gradient descent update:

$$U_{t+1} = \text{Gram-Schmidt}(U_t + \eta_t \mathbf{x}_t \mathbf{x}_t^T U_t) \quad (3)$$

where η_t is the learning rate at the t^{th} iteration and the Gram-Schmidt orthogonalization gives an orthogonal basis for its input. It has been shown that under certain conditions on distribution of x and the step size η , the algorithm converges with a probability 1. It has also been shown this algorithm enjoys a fast $O(\frac{1}{t})$ iteration complexity. Note that in this algorithm we have at every iteration a $d \times k$ matrix which is orthogonal but the cost of performing the Gram-Schmidt orthogonalization is $O(kd^2)$. But Arora et al. [2012] show that the normalization step doesn't affect the correctness of the solution and thus the algorithm can be made efficient costing $O(kd)$ operations per iteration.

2.2 Incremental PCA

The basic premise behind this algorithm is to use an incremental algorithm for the eigen-decomposition of the matrix of the second order moments. In this algorithm, we process one example at a time, resulting in a rank-one update to the empirical second-moment matrix at each iteration. Formally, we can write the update as :

$$C^{(t)} = \mathbf{P}(C^{(t-1)} + \mathbf{x}_t \mathbf{x}_t^T) \quad (4)$$

where $C^{(t)}$ denotes the empirical co-variance matrix at the t^{th} iteration and $\mathbf{P}$ is the operator which projects onto the set of rank- k matrices with respect to the spectral norm.

Instead of a naive implementation of (4), we can use an efficient implementation of the algorithm as described in Arora et al. [2012]. Suppose $C^{(t-1)}$ is rank- l approximation of the co-variance matrix and its eigen-decomposition is $C^{(t-1)} = U S U^T$ where $U \in \mathbb{R}^{d \times l}$ and $U^T U = I$. Given a new observation x_t , let $\hat{x} = U^T x_t$ be the coefficients of the projection of x_t on the columns of U and $x_{\perp} = x_t - U U^T x_t$ be its orthogonal component. Then the update can be written as:

$$\begin{bmatrix} U & \frac{x_{\perp}}{\|x_{\perp}\|_2} \end{bmatrix} \begin{bmatrix} S + \hat{x} \hat{x}^T & \|x_{\perp}\|_2 \hat{x} \\ \|x_{\perp}\|_2 \hat{x}^T & \|x_{\perp}\|_2^2 \end{bmatrix} \begin{bmatrix} U^T \\ \frac{x_{\perp}^T}{\|x_{\perp}\|_2} \end{bmatrix} \quad (5)$$

Let the middle matrix in the above equation be renamed as Q . If the eigen-decomposition of $Q = U' S' U'^T$, then the eigen-decomposition of $\tilde{C}^{(t)}$ is $\tilde{U} \tilde{S} \tilde{U}^T$ where:

$$\tilde{U} = \begin{bmatrix} U & \frac{x_{\perp}}{\|x_{\perp}\|_2} \end{bmatrix} U' ; \tilde{S} = S' \quad (6)$$

If the rank of $\tilde{C}^{(t)}$ goes beyond k , then we only keep the top k eigenvalues and eigenvectors in $\tilde{S}$ and $\tilde{U}$ respectively. The major computational cost in this algorithm comes from the matrix multiplication in (6) which has a cost of $O(k^2 d)$ which is generally small since in practice a lot of times $k \ll d$. An important thing to note is that this algorithm may fail to converge - consider data from the distribution which samples $(3, 0)$ with probability $1/3$ and $(0, 2)$ with probability $2/3$. In this case the algorithm will converge to $(1, 0)$ with probability $5/9$ even though the maximal eigenvector is $(0, 1)$.

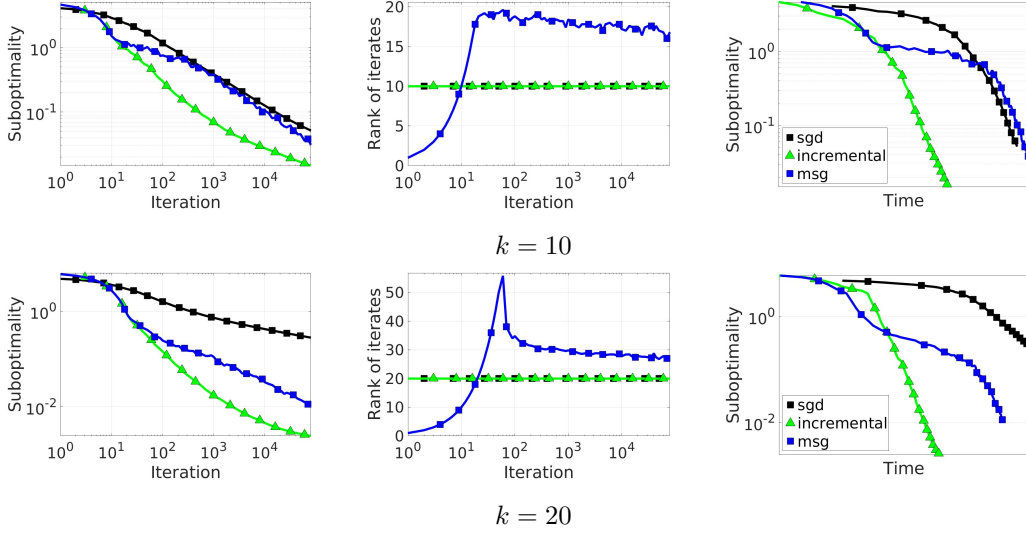


Figure 1: Comparison of Oja’s Algorithm (SGD in figure), incremental PCA and MSG on the synthetic dataset for $k = 10$ (above) and $k = 20$ (below). The first graph shows the suboptimality vs number of iteration, the second the rank of the intermediate iterates vs number of iterations and the last one shows suboptimality as a function of running time.

2.3 MSG

This algorithm first uses a re-parameterization to make the PCA objective function convex. If $M = UU^T$, the PCA problem can be reformulated as:

$$\begin{aligned} \max \mathbb{E}[\mathbf{x}^T M \mathbf{x}] \\ \text{s.t. } M \in \mathbb{R}^{d \times d}, \lambda_i(M) \in \{0, 1\}, \text{rank} M = k. \end{aligned} \quad (7)$$

where $\lambda_i(M)$ is the i^{th} eigenvalue of M . This objective function is convex (linear in fact) but since the constraints in (7) are still non-convex, we take a convex hull of the feasible region:

$$\begin{aligned} \max \mathbb{E}[\mathbf{x}^T M \mathbf{x}] \\ \text{s.t. } M \in \mathbb{R}^{d \times d}, \lambda_i(M) \in [0, 1], \text{rank} M = k. \end{aligned} \quad (8)$$

The update rule of this algorithm reduces to performing SGD on (8) :

$$M_{t+1} = \mathcal{P}(M_t + \eta_t \mathbf{x}_t \mathbf{x}_t^T) \quad (9)$$

where $\mathcal{P}(\cdot)$ is the appropriate operator for projection onto the feasible set. To account for the suboptimal solutions for problem (8) which are not a feasible point for problem (7) (i.e., not rank k), we perform the following procedure : If $M = \frac{1}{T} \sum_{t=1}^T M^{(t)}$ is the average iterate we sample a rank- k solution from $M^{(t)}$ using the rounding procedure as described in Warmuth and Kuzmin [2007].

The analysis in Mianjy and Arora [2018] shows that this algorithm has a convergence rate of $O(\frac{\log T}{\sqrt{T}})$ which is less than Oja’s algorithm. As shown in Arora et al. [2013], we can perform this algorithm by maintaining an up-to-date eigendecomposition and then performing a rank-one eigen-update similar to that in the incremental algorithm. This gives a per iteration memory requirement of $O(dk_t)$ and $O(dk_t^2)$ operations where k_t is the rank of the intermediate iterate.

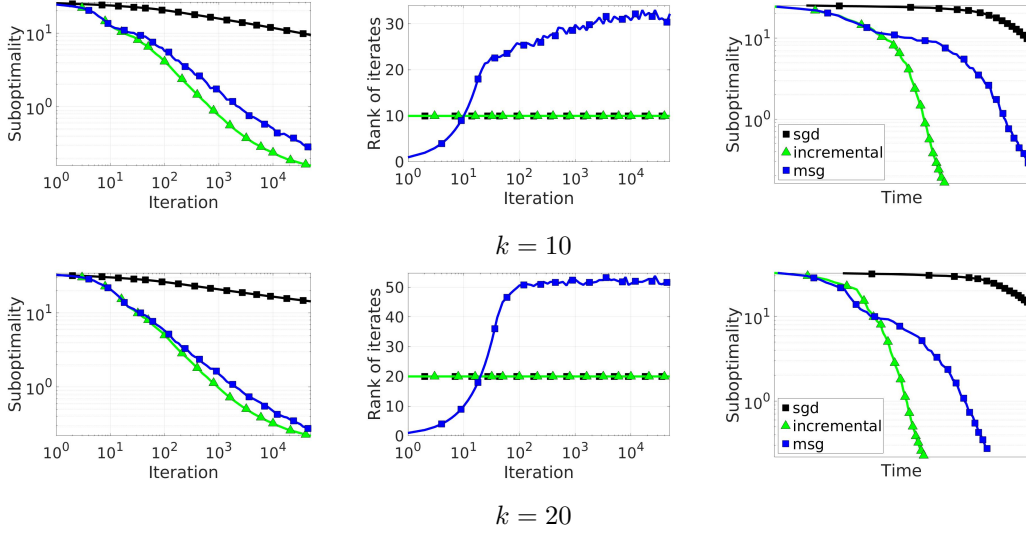


Figure 2: Comparison of Oja’s Algorithm (SGD in figure), incremental PCA and MSG on the MNIST dataset for $k = 10$ (above) and $k = 20$ (below). The first graph shows the suboptimality vs number of iteration, the second the rank of the intermediate iterates vs number of iterations and the last one shows suboptimality as a function of running time.

3 Experimental Results

In this section, we show empirical results for the three algorithms described in Section 2 on a synthetically created dataset and the MNIST dataset. The synthetic data is drawn from a 100 dimensional zero mean multivariate Gaussian distribution with an exponential decay in the spectrum of the covariance matrix. The dataset consists of 8k samples which are used for training and is tested on separate 1k samples. The MNIST dataset is a famous dataset consisting of handwritten digits where each image is represented as a 28×28 matrix. We concatenate all the pixel values to get a single 784 dimensional vector. We choose 50k samples for training and 10k samples for held-out testing.

For the synthetic dataset, we use $\eta_0 = 2$ whereas for the MNIST dataset we use $\eta_0 = 0.1$ where the learning rate is $\frac{\eta_0}{\sqrt{t}}$ (this was picked after tuning on a separate validation dataset).

3.1 Analysis

Figure 1 and 2 shows the results on the synthetic dataset and MNIST dataset respectively, for the size of the subspace $k = 10$ and $k = 20$. As we can see, for both the datasets the incremental PCA algorithm performs much better both in terms of overall run-time and faster convergence in terms of number of iterations. For the synthetic dataset, the MSG algorithm and Oja’s algorithm perform similar for smaller values of k in terms of suboptimality per iteration but MSG algorithm fares much better for larger values of k . On the more practical MNIST dataset, MSG algorithm consistently outperforms Oja’s algorithm in per iteration complexity and overall run-time.

The second graph for each dataset shows the rank of the intermediate iterates. By construction of the algorithms, the rank of the intermediate iterates in Oja’s algorithm and incremental PCA is fixed and is equal to the dimensionality k to be reduced to. For MSG Algorithm, we observe that the rank of the iterates increases and then almost plateaus but is still considerably larger than k .

As shown in Section 2, the per iteration complexity of MSG is $O(dk_t^2)$ and thus depends on the rank of the intermediate iterates k_t . To tackle this, Arora et al. [2013] propose capped MSG where we have an extra constraint that the rank of the intermediates is less than some constant K which has been shown to perform much better simple MSG. Also, Mianjy and Arora [2018] suggest an alternative variant where they impose l_1 regularization and prove its effect on controlling the rank of the iterates.

4 Conclusion

In this paper, we compared three different algorithms for stochastic PCA both in terms of theoretical guarantees and empirically on multiple datasets. We showed that incremental algorithm performs very well in practice but could potentially fail to converge as shown through an example of a carefully curated dataset. On the other hand, MSG algorithm can perform almost well and can be made more efficient with some variations such as capped MSG [Arora et al., 2013] or by adding regularization [Mianjy and Arora, 2018].

References

- Erkki Oja. Simplified neuron model as a principal component analyzer. *Journal of Mathematical Biology*, Nov 1982.
- Raman Arora, Andy Cotter, and Nati Srebro. Stochastic optimization of pca with capped msg. In *Advances in Neural Information Processing Systems 26*. 2013.
- Poorya Mianjy and Raman Arora. Stochastic PCA with ℓ_2 and ℓ_1 regularization. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- Zeyuan Allen-Zhu and Yuanzhi Li. First efficient convergence for streaming k-pca: A global, gap-free, and near-optimal rate. *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 487–492, 2017.
- Raman Arora, Andrew Cotter, Karen Livescu, and Nathan Srebro. Stochastic optimization for pca and pls. *2012 50th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 861–868, 2012.
- Manfred K Warmuth and Dima Kuzmin. Randomized pca algorithms with regret bounds that are logarithmic in the dimension. In *Advances in Neural Information Processing Systems*. 2007.